

TRYHACKME - RELATÓRIO TÉCNICO

Fundamentos de HTTP

URLs, Requisições, Métodos, Códigos de Status, Cabeçalhos e Cookies

Autor: Amilcar Junior

Plataforma: TryHackMe

Data: 26 de agosto de 2026

Sumário

1. Introdução ao HTTP e HTTPS	3
2. Anatomia de uma URL	4
3. Requisições e Respostas HTTP	5
4. Métodos e Códigos de Status	6
5. Cabeçalhos e Cookies	7
6. Exercício Prático	8
6.1 GET e parâmetros de URL	8
6.2 GET /room e POST /login	9
6.3 PUT e DELETE	10
7. Conclusão	11

1. Introdução ao HTTP e HTTPS

Este relatório documenta a conclusão do módulo de Fundamentos de HTTP na plataforma TryHackMe. O HTTP (Hypertext Transfer Protocol) define as regras usadas por navegadores, APIs e servidores para solicitar e entregar recursos como páginas HTML, imagens, arquivos, vídeos e dados estruturados.

O protocolo opera no modelo cliente-servidor: o cliente envia uma requisição informando o recurso e a ação desejada; o servidor processa o pedido e devolve uma resposta com código de status, cabeçalhos e, quando aplicável, um corpo de dados.

1.1 HTTP x HTTPS

Característica	HTTP	HTTPS
Proteção do tráfego	Dados trafegam sem criptografia.	Dados são protegidos por TLS.
Porta padrão	80	443
Identidade do servidor	Não oferece validação criptográfica.	O certificado digital ajuda a validar o servidor.
Risco principal	Interceptação e alteração do tráfego.	Reduz espionagem e ataques Man-in-the-Middle quando configurado corretamente.

Relevância para Segurança da Informação

Entender HTTP permite analisar tráfego web, reconhecer dados sensíveis expostos, interpretar alertas de WAF ou proxy e investigar requisições suspeitas. HTTPS protege o canal, mas não torna automaticamente a aplicação segura.

A base do protocolo também aparece em ferramentas de segurança como Burp Suite, OWASP ZAP, Wireshark e proxies corporativos. Dominar sua estrutura é pré-requisito para troubleshooting, análise de logs, resposta a incidentes e testes de aplicações web.

2. Anatomia de uma URL

Uma URL (Uniform Resource Locator) descreve como e onde um recurso pode ser acessado. Ela pode reunir protocolo, credenciais, domínio, porta, caminho, parâmetros e um fragmento. Nem todos esses componentes aparecem em todas as requisições.

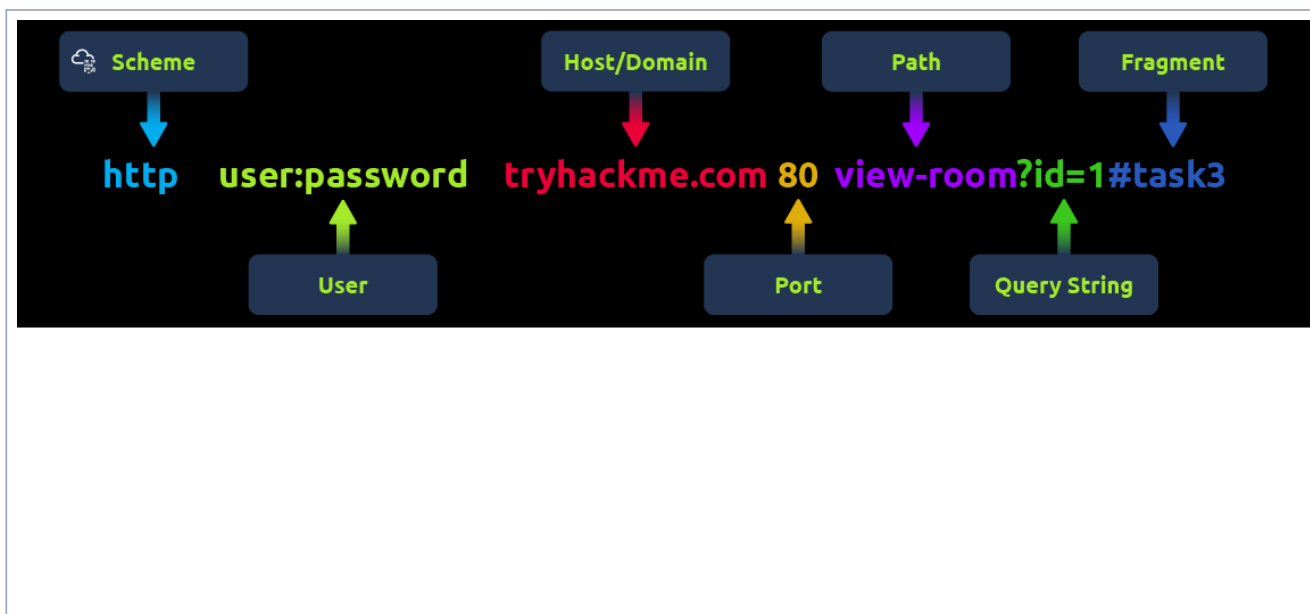


Figura 1: Componentes de uma URL completa: esquema, usuário, host, porta, caminho, query string e fragmento.

Componente	Função
Esquema	Define o protocolo de acesso, como HTTP, HTTPS ou FTP.
Usuário	Pode carregar credenciais em serviços legados; não é recomendado expor senhas na URL.
Host/Domínio	Identifica o servidor por nome de domínio ou endereço IP.
Porta	Indica o serviço de destino; normalmente 80 para HTTP e 443 para HTTPS.
Caminho	Aponta para o recurso ou rota solicitada, como /blog ou /login.
Query string	Envia parâmetros após '?', por exemplo /blog?id=1.
Fragmento	Aponta para uma seção da página após '#'; normalmente não é enviado ao servidor.

3. Requisições e Respostas HTTP

Uma comunicação HTTP começa com uma linha de requisição. Em GET / HTTP/1.1, GET é o método, / representa o recurso solicitado e HTTP/1.1 informa a versão do protocolo. Depois aparecem os cabeçalhos e, quando necessário, um corpo com dados.

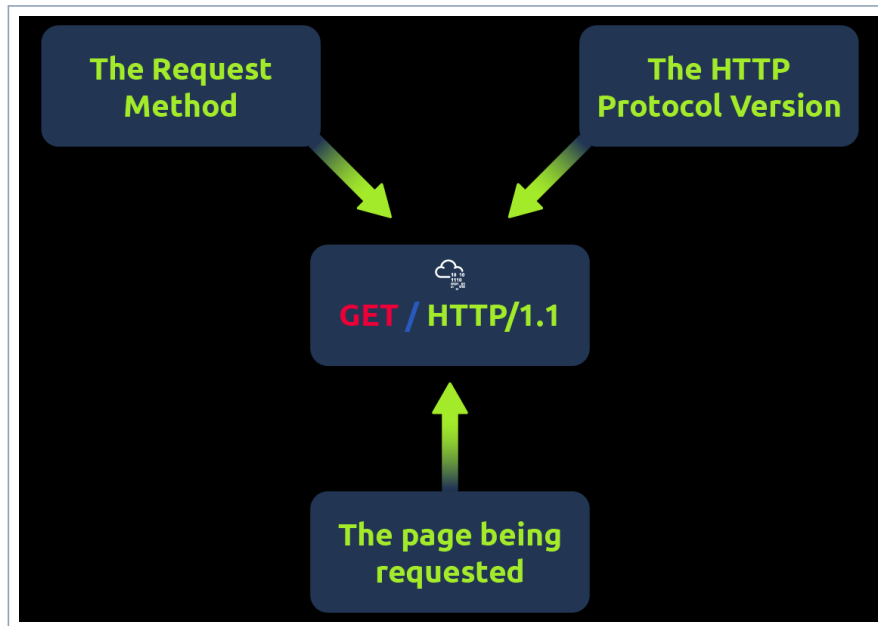


Figura 2: Estrutura da linha inicial de uma requisição HTTP.

Exemplo de requisição

```
GET / HTTP/1.1
Host: tryhackme.com
User-Agent: Mozilla/5.0 Firefox/87.0
Referer: https://tryhackme.com/
```

Exemplo de resposta

```
HTTP/1.1 200 OK
Server: nginx/1.15.8
Content-Type: text/html
Content-Length: 98

<html>...</html>
```

Leitura técnica da resposta

O código 200 indica sucesso. Server identifica o software do serviço, Content-Type informa o tipo do conteúdo e Content-Length indica o tamanho do corpo. Cabeçalhos e versões expostos também podem ajudar na identificação de tecnologias durante uma análise de segurança.

4. Métodos e Códigos de Status

4.1 Métodos HTTP

Método	Objetivo	Exemplo no laboratório	Observação
GET	Consultar um recurso.	GET /blog?id=1	Não deve alterar o estado do recurso.
POST	Enviar dados e criar/processar registros.	POST /login	Os dados podem seguir no corpo da requisição.
PUT	Atualizar ou substituir um recurso.	PUT /user/2	Normalmente é idempotente.
DELETE	Remover um recurso.	DELETE /user/1	Exige controle de autorização rigoroso.

4.2 Faixas de códigos de status

Faixa	Categoria	Interpretação
100-199	Informação	A requisição continua em processamento.
200-299	Sucesso	O servidor processou a solicitação.
300-399	Redirecionamento	O recurso deve ser buscado em outro local.
400-499	Erro do cliente	Há problema na requisição, autenticação ou autorização.
500-599	Erro do servidor	O serviço falhou ao processar uma requisição válida.

4.3 Códigos comuns

Código	Significado	Código	Significado
200 OK	Operação concluída.	201 Created	Recurso criado.
301/302	Redirecionamento permanente/temporário.	400 Bad Request	Requisição inválida.
401 Unauthorized	Autenticação necessária.	403 Forbidden	Acesso negado.
404 Not Found	Recurso inexistente.	405 Method Not Allowed	Método não aceito.
500 Internal Error	Falha interna do servidor.	503 Unavailable	Serviço indisponível.

5. Cabeçalhos e Cookies

Cabeçalhos transportam metadados que orientam o processamento da requisição e da resposta. Eles informam o host, o navegador, o tamanho e o tipo do conteúdo, mecanismos de compressão, políticas de cache e informações de sessão.

Direção	Cabeçalho	Finalidade
Requisição	Host	Seleciona o site correto em servidores que hospedam vários domínios.
Requisição	User-Agent	Identifica o cliente, navegador e versão.
Requisição	Content-Length	Informa o tamanho dos dados enviados.
Requisição	Cookie	Devolve ao servidor os dados de sessão armazenados.
Resposta	Set-Cookie	Solicita que o cliente armazene um cookie.
Resposta	Content-Type	Define o formato retornado: HTML, JSON, imagem, PDF etc.
Resposta	Cache-Control	Controla por quanto tempo o conteúdo pode permanecer em cache.

O HTTP é sem estado: cada requisição é independente. Cookies permitem que o servidor reconheça o cliente em acessos posteriores, mantendo sessões autenticadas e preferências. Em aplicações reais, cookies de sessão devem usar proteções como Secure, HttpOnly e SameSite.



Figura 3: Fluxo de criação e reutilização de um cookie de sessão.

6. Exercício Prático

6.1 GET e parâmetros de URL

No navegador de laboratório do TryHackMe, foram construídas requisições manualmente para observar como método, caminho, parâmetros, cabeçalhos e respostas se relacionam. O primeiro teste solicitou a raiz do domínio com GET.

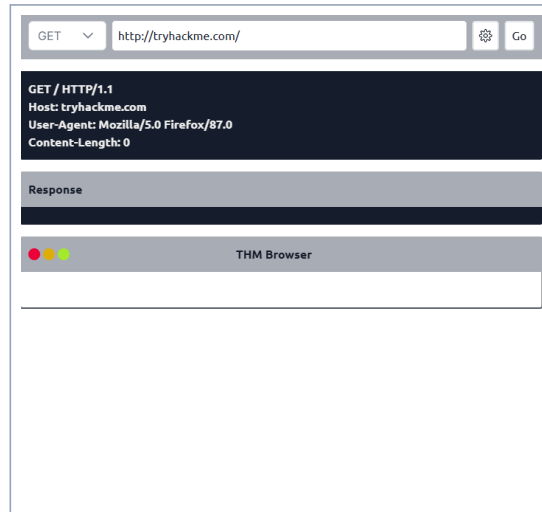


Figura 4: Requisição GET para a raiz de tryhackme.com, com a linha GET / HTTP/1.1.

Em seguida, foi utilizado o parâmetro `id=1` na query string da rota `/blog`. A resposta 200 OK retornou o conteúdo do artigo 1, demonstrando como parâmetros podem selecionar registros específicos no servidor.

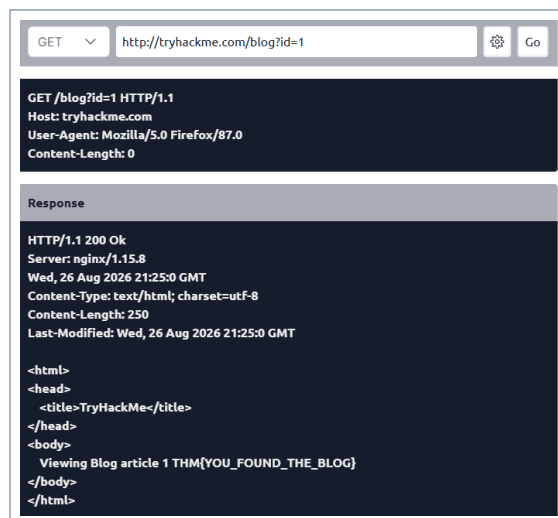


Figura 5: GET /blog?id=1 e resposta HTTP 200 com o artigo solicitado.

Ponto de atenção

Parâmetros previsíveis, como `id=1`, devem ser protegidos por validação e autorização. Sem esses controles, a aplicação pode ficar vulnerável a acesso indevido a objetos, conhecido como IDOR/BOLA.

6.2 GET /room e POST /login

A requisição GET para /room retornou a página da sala e confirmou a leitura correta do caminho solicitado. O corpo HTML incluiu a flag do exercício, enquanto os cabeçalhos registraram servidor, tipo de conteúdo e tamanho da resposta.

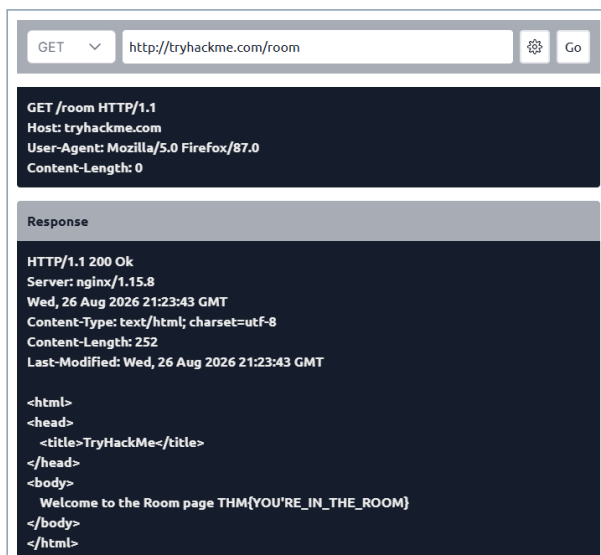


Figura 6: GET /room com resposta 200 OK e conteúdo HTML da sala.

Para autenticação, foi enviado um POST para /login com dados no formato application/x-www-form-urlencoded. Diferentemente do GET, o conteúdo foi transmitido no corpo da requisição, representando um fluxo típico de formulário web.

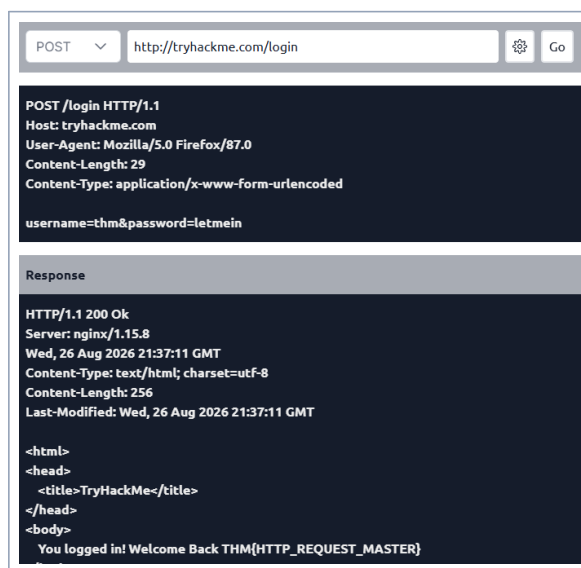


Figura 7: POST /login com credenciais no corpo e confirmação de login na resposta.

Análise de segurança

Credenciais nunca devem trafegar por HTTP simples. Em um ambiente real, o login deve usar HTTPS, limitar tentativas, evitar mensagens excessivamente detalhadas e tratar a sessão com cookies seguros.

6.3 PUT e DELETE

O método PUT foi usado para atualizar o usuário 2. O corpo username=admin foi enviado ao servidor, que respondeu com 200 OK e confirmou a alteração. Esse teste demonstra como APIs e aplicações recebem atualizações de recursos existentes.

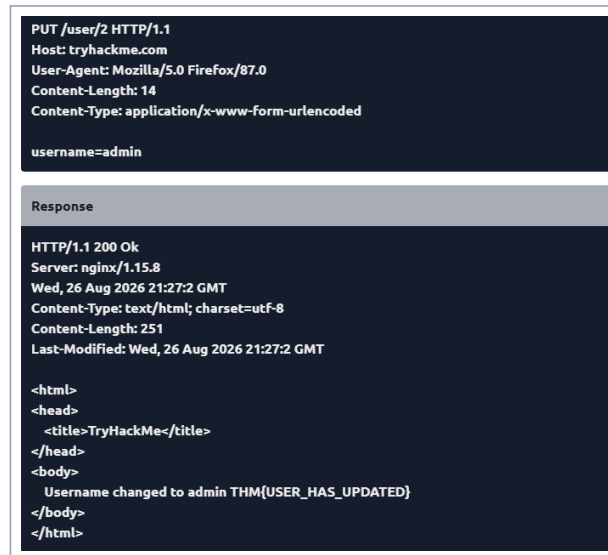


Figura 8: PUT /user/2 atualizando o nome de usuário para admin.

Por fim, o método DELETE foi enviado para /user/1. A resposta confirmou a exclusão do registro. Como se trata de uma operação destrutiva, a aplicação deve validar autenticação, autorização, escopo do usuário e integridade do identificador antes de executar a ação.

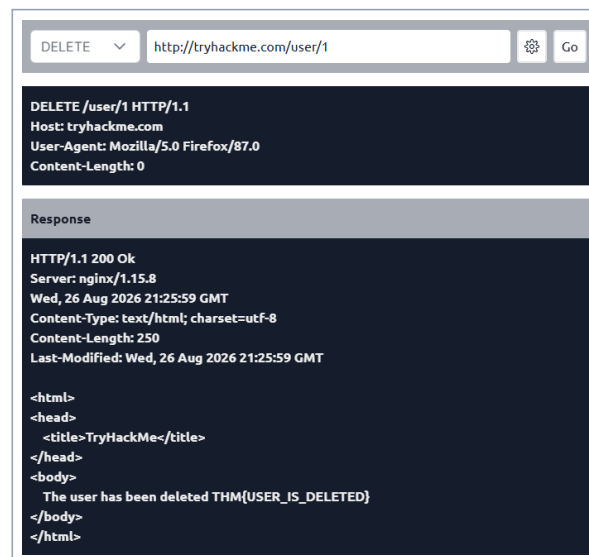


Figura 9: DELETE /user/1 e confirmação de exclusão do usuário.

Controles recomendados

Operações PUT e DELETE devem aplicar autorização no servidor, registro de auditoria, validação de entrada e proteção contra CSRF quando acionadas por navegador. Ocultar o botão no front-end não é controle de segurança.

7. Conclusão

Este laboratório consolidou a estrutura fundamental do HTTP, desde a composição de URLs até o ciclo completo de requisição e resposta. Foi possível interpretar métodos, caminhos, cabeçalhos, códigos de status, corpos de mensagem e cookies, além de diferenciar o transporte sem criptografia do HTTP da proteção oferecida pelo HTTPS.

Na etapa prática, foram executadas requisições GET, POST, PUT e DELETE no ambiente do TryHackMe. Os testes mostraram como parâmetros de URL selecionam recursos, como formulários enviam dados, como registros são atualizados e como operações destrutivas são solicitadas ao servidor.

Sob a ótica de Segurança da Informação, os resultados reforçam que cada elemento do protocolo cria pontos de observação e controle: URLs podem expor identificadores, cabeçalhos revelam contexto técnico, cookies sustentam sessões e métodos de alteração exigem autorização rigorosa. O domínio desses conceitos apoia atividades de SOC, MSS, Blue Team, troubleshooting, análise de logs e segurança de aplicações web.

Competências demonstradas

Leitura de requisições e respostas HTTP; interpretação de métodos e códigos de status; análise de URLs, parâmetros e cabeçalhos; entendimento de cookies e sessões; identificação de riscos básicos em aplicações web; documentação técnica com evidências reproduzíveis.

Próximos passos recomendados para o portfólio

Próximo laboratório	Objetivo profissional
Análise de tráfego HTTP/HTTPS no Wireshark	Relacionar o protocolo à investigação de rede.
Interceptação controlada com Burp Suite	Visualizar e modificar requisições em ambiente autorizado.
OWASP Top 10 em laboratório	Aplicar HTTP a falhas reais de autenticação, autorização e entrada.
Logs de servidor e WAF	Treinar triagem de eventos para rotina de SOC/MSS.